

~\Desktop_AI_Lab_\NOTES.md

Comprehensive Study Notes: AI Engineering & Production LLM Applications

This guide contains structured notes, concrete examples, analogies, code snippets, active recall questions, and revisions derived from the course material. All video timestamps have been removed.

Module 1: Your First LLM API Call

1. Virtual Environments (venv)

A **virtual environment** is an isolated, sandboxed directory that manages dependencies and Python libraries for a specific project.

- **Why use it?** It prevents dependency conflicts. For example, if **Project A** requires an older version of a tool/library while **Project B** requires a newer one, virtual environments keep their dependencies separated.
- **Workflow:**
 1. Create a dedicated directory.
 2. Initialize the environment (using tools like uv).
 3. Activate the environment to contain all development activity within that specific folder.

2. Anatomy of an LLM API Call

To interact with an LLM programmatically (e.g., using Groq), you need several key components:

- **API Key:** Acts as a secure password/credential.

[!IMPORTANT] **Security Guardrail:** Never hardcode your API key directly in your script. Use a `.env` file to store secrets safely.

- **Client Initialization:** Establish a connection to the server by registering your client using your API key.
- **Model Selection:** Specify the exact model name. Model names are strict (e.g., `Llama 3.3 70B`). Spelling errors here result in an immediate execution failure.

- **Messages:** The input provided to the model. An API call requires a list of messages, where each message is defined by a role and content.

Message Roles

- **System:** Sets the behavior, constraints, and personality for the entire conversation.
- **User:** The input prompt provided by the human user.
- **Assistant:** The model's previous response; essential for maintaining conversational context.

3. Handling Responses

When you receive a response from an LLM API, it returns a large object containing both the response text and rich metadata:

- **Choices:** An array containing the model's generated answers. You access `choices[0]` to retrieve the primary response.
- **Usage:** Tracks how many tokens were consumed for the request (prompt tokens, completion tokens, and total tokens), which is critical for cost management.

[!WARNING] **API Call Syntax:** A common developer mistake is misnaming keys in the message dictionary (e.g., using `prompt` instead of `content` or `message` instead of `messages`).

4. Practical Implementation Steps

1. **Initialize Environment:** Use `uv init` and `uv venv` to set up your directory.
2. **Install Dependencies:** Install libraries using your package manager (e.g., `uv add groq-python`).
3. **Authentication:** Use `load_dotenv()` to pull your API key from your `.env` file.
4. **Execute:** Pass your list of messages containing roles and content to the client.
5. **Debug:** If the code fails, comment out lines to isolate where the error occurs, and inspect the specific error message.

Active Recall Questions (Module 1)

System User Assistant

1. **Why is it bad to hardcode API keys in a script?**
Answer: It risks leaking your credentials if you share or push the code to a public repository.
2. **What is the main purpose of a virtual environment?**
Answer: To isolate project dependencies and avoid version conflicts.
3. **What are the three common roles in an LLM message list?**
Answer: System, User, and Assistant.
4. **Why does the API require a list of messages instead of a single string?**
Answer: To maintain conversational context so the model knows what was previously discussed.

5. What is a 'token' in the context of LLM usage?

Answer: A unit of text (often parts of words) used to measure model input/output and cost.

6. How do you retrieve the main text response from a Groq API object?

Answer: Via `response.choices[0].message.content`.

7. What happens if you use the wrong model name in your API call?

Answer: The call will fail because the API server cannot find the requested model.

8. What is the difference between the System role and the User role?

Answer: System sets the behavior/instructions; User provides the input or questions.

9. How do you safely manage sensitive API keys in your project?

Answer: Use an `.env` file and exclude it from version control (e.g., via `.gitignore`).

10. What does `load_dotenv()` do in your Python script?

Answer: It loads variables defined in a local `.env` file into the system's environment variables.

11. If you get an error in your code, what is a suggested way to debug it?

Answer: Comment out lines of code to isolate the specific line causing the issue.

12. What does '70B' usually refer to in a model name like 'Llama 3.3 70B'?

Answer: 70 Billion parameters, which generally correlates to model complexity and power.

Module 2: System Roles & Temperature Tuning

1. System Roles

The System Role is the most powerful lever for defining the model's identity and behavioral boundaries.

- **Contextual Identity:** It establishes the relationship between the user and the LLM (e.g., "You are a mentor" vs. "You are a coding assistant").
- **Professional Boundaries:** A single prompt (e.g., "I love you") can result in vastly different responses depending on the system role defined (e.g., a romantic partner vs. a professional manager).
- **Real-World Application:** In corporate AI agent setups, you can assign different system roles to different instances of the same LLM (e.g., one agent acting as a Senior Architect, another as a Junior Developer) to ensure that responses align with specific responsibilities.

2. Temperature

Temperature controls the LLM's "personality" or the randomness of its predictions (on a scale from 0 to 2).

- **Temperature = 0 (Deterministic):** The model always chooses the most probable, factual, and consistent next word. Use this for factual Q&A, coding, classification, or data extraction where accuracy is priority.

- **Temperature = 1+ (Creative):** The model takes more risks, favoring less obvious, creative, or random words. Use this for creative writing, brainstorming, or naming a brand.
- **Warning:** LLMs are prediction engines, not truth-telling machines. A higher temperature doesn't make it "smarter"; it just makes it more unpredictable.

Active Recall Questions (Module 2)

1. **How does the system role influence an LLM's response to an identical user input?**
Answer: It changes the context, tone, and perspective of the AI.
2. **What is the primary difference between a "System Role" and "Temperature"?**
Answer: System role = Persona/Rules; Temperature = Randomness/Creative variance.
3. **Why should you avoid high temperature for a medical AI agent?**
Answer: A medical agent requires factual accuracy; high temperature introduces unnecessary and potentially harmful randomness.
4. **If you set temperature to 0, what is the model attempting to do?**
Answer: It plays it safe by picking the most probable next token (the most "correct" sounding word).
5. **What happens if you try to set the temperature to 3?**
Answer: The API will typically throw an error, as most models operate within a 0–2 range.
6. **In what order should you pass messages to the API?**
Answer: The system message must be passed first.

Module 3: Understanding Tokens in LLMs

1. The Token Concept

Tokens = NUMERICAL REPRESENTATION = COMMON REUSABLE BUILDING BLOCKS OF WORDS

LLMs process numbers, not natural language. To bridge this gap, text must be converted into numerical representations (tokens).

- **Why not character-level processing?** It leads to excessively large inputs. A simple "Hello" would require a massive numeric overhead, making the model computationally inefficient and memory-intensive.
- **Why not word-level processing?** Using a fixed dictionary (e.g., Oxford Dictionary with 600,000 words) fails to handle slang, misspellings, proper nouns (e.g., "Pratyush"), and compound words.
- **The Token Solution:** Tokens are common, reusable building blocks of words.
 - Common words like "the" or "in" are often treated as single tokens.
 - Complex or rare words are broken down into smaller, recognizable tokens (e.g., the proper noun "Pratyush" might be split into "Pro", "ty", "ush").
- **The LEGO Block Analogy:** Think of words as complex toys (like a LEGO castle) and tokens as the individual LEGO bricks. The tokenizer breaks the castle down into standard

blocks so that the model's neural network only needs to learn how to put standard bricks together.

2. Tokenization and Cost

LLMs charge by the number of tokens, not the word count.

- **Input Tokens:** The prompt text sent to the LLM.
- **Output/Completion Tokens:** The response generated by the LLM.
- **Total Cost** = Input Tokens + Output Tokens.
- **The “Hi” Cost Overhead:** The raw text "Hi" is only 1 token, but when you send it via an API call, it gets wrapped in system overhead, message role keys, and formatting boilerplate, costing more tokens (e.g. 36 tokens) at the API level.

3. Production Control: Managing API Costs

- **max_tokens:** An “emergency brake” parameter. Limits how many tokens the model can generate to prevent runaway costs from hallucination loops.
- **finish_reason:** A field in the API response that indicates how the generation ended:
 - stop: The model finished naturally.
 - length: The model was forced to stop because it hit the max_tokens limit.
- **Reserved Names Warning:** Do not name your Python file token.py (or token.pi). This causes circular import errors because the name conflicts with internal Python libraries.

```
# Retrieving usage stats in Python
response = client.chat.completions.create(...)
usage = response.usage
print(f"Input tokens: {usage.prompt_tokens}")
print(f"Completion tokens: {usage.completion_tokens}")
```

Active Recall Questions (Module 3)

1. Why is character-level processing inefficient for LLMs?

Answer: It creates massive numeric overhead for even short inputs.

2. What is the fundamental problem with using a fixed dictionary for word-level processing?

Answer: It cannot handle slang, misspellings, or words outside the static dictionary.

3. Why should you avoid naming your Python script token.py?

Answer: It is a reserved name in Python and causes circular import errors.

4. What does the finish reason of “length” indicate?

Answer: The generation was cut off because it hit the max_tokens limit.

Module 4: Structured Output (Pydantic + JSON)

1. The Raw Text Parse Problem

LLMs naturally output plain text strings. Parsing these strings (e.g., extracting emails, phone numbers, or status codes) using regex is error-prone. In real-world AI applications, 90% of the time, the LLM output is consumed by another software component (written in Python, C++, etc.), not a human. If the LLM changes its phrasing or adds filler text (e.g., "Sure! Here is your requested info."), regex-based parsers break.

Pydantic : Data Validation Library
Shape of data

2. JSON Mode and Pydantic

- **JSON:** The industry standard for inter-service communication.
- **Pydantic:** A Python data-validation library that allows you to define the exact shape of data you expect from an LLM.

Feature	Unstructured Output	Structured Output (JSON)
Readability	High (Human)	Moderate (Human/Machine)
Parsing	Extremely Hard (Manual Regex)	Trivial (Standard JSON Libraries)
Reliability	Low (Varies dynamically)	High (Strict Schema Enforcement)

Explicit "JSON" Mention Warning: If you don't explicitly mention "JSON" in your system prompt, the LLM may fail to use the requested format even if the response_format parameter is set to JSON mode.

Pydantic Constraints: Use Literal types to restrict LLM outputs to allowed values (e.g., constraining a classifier to output only "positive", "neutral", or "negative").

Active Recall Questions (Module 4)

1. Why is temperature=0 recommended for data extraction?

Answer: It minimizes randomness and ensures consistent, schema-compliant, and reproducible outputs.

2. What happens if you forget to mention 'JSON' in the system prompt in JSON mode?

Answer: The LLM might ignore the schema, hang, or fail to return a proper JSON object.

3. What is the role of the BaseModel in Pydantic?

Answer: It defines the required structure (fields and types) of the data to validate.

Module 5: Resume-to-JD Matcher Mini-Project

This project matches resumes against a specific Job Description (JD) using structured data extraction.

1. Project Architecture & Workflow

- **Part 1: Parsing the Job Description (JD):** Use a Pydantic schema to define a JobDescription model containing: Role, Required Skills, Preferred Skills, Minimum Experience, Educational Requirements, and Responsibilities.
- **Part 2: Defining the Resume Schema:** Create a hierarchical class model:
 - Experience class (Company name, Role, Duration, Description, Skills used).
 - Resume class (Name, Contact, Total Experience, Skills [List], Experiences [List of Experience], Projects, Certifications).

[!TIP] **Resilience Tip:** Set optional fields to None to prevent the parser from crashing when a resume is missing specific sections.

- **Part 3: Resume Parsing (PDF/Word):** Implement read_pdf(), read_docs(), and a routing function read_resume().
- **Part 4: Intelligent Extraction:** Pass the extracted text to an LLM at temperature=0 with instructions to strictly use the Resume schema.

[!WARNING] **Do not ask the LLM to “hallucinate” data.** If information is missing in the text, it should return an empty list or None.

- **Part 5: Matching & Scoring:** Pass the structured JD and Resume JSON objects to the LLM to get a MatchResult class: Score (0–100), details (Strengths/Weaknesses, Missing Skills), and a Final Verdict.
- **Rate Limiting:** Implement time.sleep(5) between API calls when looping through a directory of resumes to avoid Rate Limiting or Denial of Service (DoS) errors.

Module 6: Production-Grade Prompt Engineering

Prompts that work in ChatGPT break in code because ChatGPT has pre-loaded system prompts and conversational helper layers. Code interaction requires structured, explicit instruction sets.

1. The 6-Section Anatomy of a Production Prompt

1. **Role:** Define the system's professional domain/persona (e.g., “*You are a support assistant at a laptop company*”).
2. **Task:** Clearly define what the model must execute.
3. **Constraints:** Set strict boundaries (e.g., “*Use only these 3 categories: Billing, Technical, Return*”).
4. **Format:** Specify the target output layout (e.g., “*Answer in one word only*”).
5. **Examples (Few-Shot):** Provide explicit input-output pairs to guide performance.
6. **Fallback:** Define output instructions if the input cannot be processed (e.g., “*If query is unrelated, output: Other*”).

- **Fallback Warning:** Without a fallback directive, an LLM will force a classification even on irrelevant inputs (e.g., classifying a breakup text as a 'Billing' issue).

2. Core Prompting Rules

- **Positive Instructions:** Autoregressive models generate text based on tokens present in their context. Avoid negative constraints like "Do not write a greeting", which insert the token "greeting" into the context. Instead, write "Begin response directly with the JSON output".
- **XML Tags vs. Markdown:** Markdown is ideal for simple prompts. XML tags (e.g., `<instructions>...</instructions>`) are highly distinct and help large models separate instruction contexts from user inputs.

Module 7: ReAct Intro to AI Agents

1. What is ReAct?

CAN PERFORM TASKS and USE TOOLS

ReAct stands for **Reasoning + Acting**. It is a design pattern used to transition from simple LLMs (which only generate text) to AI Agents (which can perform tasks and use tools).

- **Simple LLM:** Answers questions using static training data.
- **AI Agent:** Uses external tools (APIs, databases) to solve problems.

2. The Problem

LLMs are trained on static historical data (e.g., data up to July 2026). They fail when asked for:

- **Real-time information:** Current weather, live stock prices, or recent news.
- **Multi-step tasks:** Complex queries requiring sequential dependencies (e.g., looking up flight prices, checking hotel availability, and calculating totals).
- **The Solution:** Give the LLM access to **Tools** (Python functions/APIs) and an **Agentic Loop** to decide which tools to call.

3. The ReAct Loop

This cycle continues iteratively until the task is complete:

1. **Thought:** The model reasons about what it needs to do next.
2. **Action:** The model chooses and calls an appropriate tool (e.g., `calculator(expression="100 * 1.08")`).
3. **Observation:** The agent runner intercepts the tool name and inputs, executes the python function, and appends the raw returned data to the conversation history.

[!IMPORTANT] **Agentic Stop:** The agent must be instructed to stop generating text immediately after writing an Action block to wait for the environment's observation.

Example Loop:

Thought: I need to check the population of Mumbai.

Action: `lookup_population(city="Mumbai")`

Observation: 20,000,000

Thought: Now I need to check the population of Bangalore.

Action: `lookup_population(city="Bangalore")`

Observation: 12,000,000

Thought: I will calculate the difference: $20,000,000 - 12,000,000$.

Action: `calculator(expression="20000000-12000000")`

Observation: 8,000,000

Final Answer: Mumbai has 8,000,000 more people than Bangalore.

Module 8: Prompt Chaining

1. ReAct vs. Prompt Chaining

- **ReAct:** The LLM dynamically decides which tools to call in an autonomous loop.
- **Prompt Chaining:** The developer defines a hardcoded, deterministic sequence of steps (pipeline) in Python.

Feature	ReAct (Reasoning + Acting)	Prompt Chaining
Control	LLM decides which tool to use next.	Developer defines the workflow/steps.
Logic	LLM handles the decision-making loop.	Sequential, deterministic pipeline.
Use Case	Agentic, autonomous task execution	Structured, multi-step pipelines

2. Why Use Prompt Chaining?

Specialist > Generalist

- ✓ **Easier Debugging:** You can inspect and log the output of every individual step.
- ✓ **Modularity:** You can modify or fix a single step without refactoring the entire system.
- **Cost Efficiency:** You can use expensive models for complex steps and cheaper, faster models (e.g., Llama 3 8B) for formatting tasks.
- ✓ **The Specialist Model:** Think of it as hiring four specialist employees (one for safety review, one for entity extraction, one for drafting, one for proofreading) rather than one generalist who gets overwhelmed.

[!WARNING] **Variable Injection:** Always use Python f-strings when passing variables between chain steps: otherwise, the LLM will not receive the dynamic content correctly.

Module 9: LLM Streaming

1. Perceived vs. Actual Latency

- **Streaming:** The LLM generates and sends response tokens one-by-one (or in small chunks) to the user.
- **Netflix Analogy:** You watch a video chunk-by-chunk as it arrives rather than downloading the entire multi-gigabyte file before playback.
- **UX Psychology:** Streaming does not make generation faster; it reduces **Time to First Token (TTFT)**.
- **The Restaurant Analogy:** If a restaurant brings your starter, main course, and dessert as they are ready, you feel the service is fast. If they wait 30 minutes to bring all three at once, you perceive the service as slow, even if the total elapsed time is identical.
- **Warning:** Never use streaming when generating JSON or structured outputs intended for machine consumption, as partial data chunks fail to parse.

NEVER use streaming in generating JSON or structured inputs

2. Implementation Code (Python + Groq API)

```
# Streaming Pattern
stream = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=messages,
    stream=True
)

for chunk in stream:
    content = chunk.choices[0].delta.content
    if content:
        print(content, end="", flush=True)
    # flush=True forces the terminal to display the character immediately
```

Module 10: AI Portfolio Challenge & Backend Architecture

This project replaces a traditional PDF resume with a conversational AI chatbot representing you professionally.

1. 12-Step Implementation Plan

- **Step 1: Create Candidate Profile:** Compile all data (Education, Skills, Projects, Experience) into a structured JSON/text file.
- **Step 2: Choose an LLM:** Select a fast inference engine (like the Groq API).
- **Step 3: Write System Prompt:** Instruct the AI to represent you professionally, be honest, and strictly use the provided profile data. ~~Forbid hallucinations.~~
- **Step 4: Build Backend API:** Use FastAPI in Python to build a web endpoint that processes and streams queries.
- **Step 5: Build Chat UI:** Create an interface using JS/HTML/CSS with a chat window, input box, send button, and auto-scroll.
- **Step 6: Connect Frontend & Backend:** Use Fetch API to pass user queries to the backend.
- **Step 7: Add Memory:** Implement conversation history (appending messages) so the AI understands context.
- **Step 8: Job Description Matching:** Allow recruiters to upload a JD and have the AI provide compatibility scores.
- **Step 9: Deploy Backend:** Host the Python API on Render or Railway.
- **Step 10: Deploy Frontend:** Host the website on Vercel.
- **Step 11: GitHub:** Upload the repository, including a README and live links.
- **Step 12: Final Polish:** Add dark mode, typing animations, and mobile responsiveness.

2. Backend Routing (FastAPI)

FastAPI routes handle URLs using decorators:

- `@app.get("/")`: Defines the home/root route to check status.
- `@app.post("/chat")`: Used for sending questions to the server.
- **Auto-reload:** Run the server via `uvicorn main:app --reload` to automatically refresh code changes.
- **API Docs:** FastAPI automatically generates interactive documentation at the `/docs` path (Swagger UI).

Module 11: Understanding RAG (Retrieval-Augmented Generation)

1. The Core Problem

Raw LLMs have three primary weaknesses: **Knowledge Cutoff**, **No Private Data access**, and **Hallucinations**.

- ✓ Knowledge cutoff
- ✓ No private data access
- ✓ Hallucinations

2. What is RAG?

RAG retrieves relevant information from a private knowledge base and inserts it as context into the prompt before generating the final answer.

Method	How it works	Why it fails
System Prompting	Putting all data directly into the system prompt.	Limited by context window length.
PDF Uploads	Uploading large documents for the LLM to read.	High token consumption; cost-prohibitive.
Modern RAG	Retrieve only relevant snippets before generating.	Optimized for accuracy, latency, and cost.

Module 12: Embeddings & Semantic Search

1. Concept of Embeddings

Embeddings are vectors (arrays of floating-point numbers) that represent semantic meaning. They act as “GPS coordinates for meaning.”

- **Feature-Based Representation Example:** Let's map food items against two features: “Sweetness” (0–10) and “Crunchiness” (0–10).
 - **Apple:** [8, 7]
 - **Cake:** [10, 2]
 - **Kurkure:** [1, 9]
 - By placing these items on a grid, a computer calculates that an Apple is closer in distance (conceptually) to Kurkure than to Cake based on crunchiness features.
- **Cosine Similarity:** The mathematical measurement of the angle between two vectors.
 - 1.0: Perfectly similar meaning.
 - ~0.5: Moderately similar.
 - Low/Negative: Unrelated.
- **Keyword vs. Semantic Search:** Keyword search is rigid. A query for “*leave policy*” misses a document titled “*Vacation Guidelines*”. Semantic search maps both to the same concept in embedding space, returning the correct document.
- **One-Hot Encoding vs. Dense Embeddings:** One-hot encoding creates sparse, high-dimensional vectors (mostly zeros) where terms are treated as independent. Embeddings create dense, low-dimensional vectors (e.g. 384 dimensions) representing rich semantic relationships.

```
# Generating embeddings in Python
```

```
from sentence_transformers import SentenceTransformer
```

```
model = SentenceTransformer('all-MiniLM-L6-v2')
embedding = model.encode("Machine learning is fun")
```

Module 13: Building Your First RAG System from Scratch

1. Technical Workflow

1. **Define Knowledge Base:** A list of documents (strings).
2. **Generate Embeddings:** Convert documents to vectors using sentence-transformers.
3. **Process Query:** Convert the user's query into an embedding.
4. **Similarity Search:** Calculate cosine similarity between the query vector and document vectors.
5. **Retrieve:** Extract the document with the highest similarity score.
6. **Thresholding:** Define a minimum similarity score (e.g. 0.6). If the best match is below the threshold, return None (preventing hallucinations).
7. **Augment & Generate:** Pass the retrieved text as context to the LLM.

2. Complete RAG Pipeline Script (Groq + Qdrant Cloud)

Below is the complete, working implementation of a RAG pipeline from your source code.

```
# =====
# PART 1 - IMPORTS AND ENVIRONMENT
# =====
import os
from dotenv import load_dotenv
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
from sentence_transformers import SentenceTransformer
from groq import Groq

# Load variables from .env
load_dotenv()
QDRANT_URL = os.getenv("QDRANT_URL")
QDRANT_API_KEY = os.getenv("QDRANT_API_KEY")
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

# =====
# PART 2 - CONNECT TO QDRANT
```

```
# =====
client = QdrantClient(
    url=QDRANT_URL,
    api_key=QDRANT_API_KEY
)
print("Connected to Qdrant Cloud!")

# =====
# PART 3 – CREATE QDRANT COLLECTION
# =====
COLLECTION_NAME = "knowledge"
EMBEDDING_SIZE = 384

# Delete collection if it already exists
if client.collection_exists(COLLECTION_NAME):
    print(f"Deleting existing collection: {COLLECTION_NAME}")
    client.delete_collection(COLLECTION_NAME)

# Create collection
client.create_collection(
    collection_name=COLLECTION_NAME,
    vectors_config=VectorParams(
        size=EMBEDDING_SIZE,
        distance=Distance.COSINE,
    ),
)
print(f"Created collection: {COLLECTION_NAME}")

# =====
# PART 4 – LOAD OUR KNOWLEDGE
# =====
with open("knowledge.txt", "r", encoding="utf-8") as f:
    documents = [
        line.strip()
        for line in f
        if line.strip()
    ]
print(f"Loaded {len(documents)} documents")

# =====
# PART 5 – CREATE EMBEDDINGS
# =====
print("Loading embedding model...")
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```

print("Embedding model ready!")
embeddings = model.encode(documents)
print(f"Generated {len(embeddings)} embeddings")

# =====
# PART 6 – CREATE QDRANT POINTS
# =====
points = []
for i, embedding in enumerate(embeddings):
    point = PointStruct(
        id=i + 1,
        vector=embedding.tolist(),
        payload={
            "text": documents[i]
        }
    )
    points.append(point)

# =====
# PART 7 – UPLOAD TO QDRANT
# =====
client.upsert(
    collection_name=COLLECTION_NAME,
    points=points
)
print(f"Uploaded {len(points)} documents to Odrant!")

# =====
# PART 8 – SEARCH QDRANT
# =====
def search(query, top_k=3):
    query_vector = model.encode(query).tolist()
    results = client.query_points(
        collection_name=COLLECTION_NAME,
        query=query_vector,
        limit=top_k,
        with_payload=True,
    ).points
    return results

# =====
# PART 9 – TEST SEARCH
# =====
query = "How many vacation days do I get?"

```

```

results = search(query, top_k=3)
print("\nSearch results:")
for result in results:
    print(f"Score: {result.score:.3f}")
    print(result.payload["text"])
    print()

# =====
# PART 10 – CONNECT TO GROQ
# =====
groq_client = Groq(
    api_key=GROQ_API_KEY
)

# =====
# PART 11 – ASK THE LLM
# =====
def ask_llm(question, context):
    prompt = f"""
Answer the question using only the information provided below.

Context:
{context}

Question:
{question}

If the answer is not present in the context, say:
"I don't know based on the provided information."
"""
    response = groq_client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ]
    )
    return response.choices[0].message.content

# =====
# PART 12 – COMPLETE RAG PIPELINE
# =====

```

```

question = "How many vacation days do I get?"
results = search(question, top_k=3)
context = "\n".join(
    result.payload["text"]
    for result in results
)
answer = ask_llm(question, context)
print("\nFinal Answer:")
print(answer)

```

Module 14: Production-Grade RAG (Vector Databases & Multi-Tenancy)

1. Why Move to a Vector Database?

Performance
Persistence
Memory Efficiency

In-memory Python lists fail as data scales.

- **Performance:** Brute force linear searches ($O(N)$) take seconds/minutes on millions of vectors. A database indexes vectors for millisecond response times.
- **Persistence:** In-memory lists disappear if the script restarts.
- **Memory Efficiency:** High dimensional indexing is optimized to prevent RAM crashes.

2. Core Concepts in Qdrant

- **Collection:** Conceptually identical to a SQL Table.
- **Point:** Conceptually identical to a SQL Row. Consists of:
 1. **ID:** Unique identifier (e.g., 1001).
 2. **Vector:** The numerical embedding array (e.g., 384 dimensions)
 3. **Payload:** The JSON metadata (e.g., {"text": "vacation info", "workspace_id": 123}).
- **Upsert:** Atomic write operation (updates if ID exists, inserts if new).

3. HNSW (Hierarchical Navigable Small World) Indexing

HNSW is an indexing algorithm that organizes data in a hierarchical graph to speed up search to $O(\log N)$.

- **Analogy:** Finding a shop in a new country. Instead of checking every shop, you locate the right city, then the correct neighborhood, and perform a local scan only in the immediate vicinity.

4. Metadata Filtering

Qdrant allows you to apply metadata constraints (filters) during HNSW traversal to restrict the search space.

- **Filter Types:**

- **must (AND):** All conditions must evaluate to true.
- **should (OR):** At least one condition must be true.
- **must_not (NOT):** Exclude points matching this condition.

```
# Pre-filtering Vector Search
client.search(
    collection_name="docs",
    query_vector=query_emb,
    query_filter=Filter(
        must=[FieldCondition(key="category", match=MatchValue(value="tech"))]
    ),
    limit=5
)
```

5. Multi-Tenant SaaS Isolation

Creating separate databases for every client is cost-prohibitive. In production (Notion/Slack AI patterns), systems use a single collection and enforce isolation using metadata filters.

- **Workspace ID Isolation:** Every point includes a `workspace_id` in its payload. Search calls enforce a strict filter:

```
{
  "filter": {
    "must": [
      { "key": "workspace_id", "match": { "value": "workspace_123" } }
    ]
  }
}
```

- **GDPR Compliance:** Delete user records dynamically by running a filter-based delete:

```
client.delete(
    collection_name="user_data",
    points_selector=models.Filter(
        must=[models.FieldCondition(key="user_id",
```

```
match=models.MatchValue(value="user_98765"))]
    )
)
```

- **Update Performance:** Use `set_payload` to update metadata. It is fast because it does not modify the vector coordinates or trigger HNSW graph rebuilding. Use `upsert` only when updating the vector coordinates themselves.

Master Quick Revision

1. **Virtual Environments:** Sandboxes dependencies (uv venv).
2. **API Roles:** `system` sets identity; `user` poses query; `assistant` saves conversation history.
3. **Temperature:** 0 for deterministic outputs; 1+ for creativity.
4. **Tokens:** Learned syllables, not words. Output tokens cost more due to sequential generation overhead.
5. **Pydantic:** Enforces schemas on raw JSON outputs at runtime (`BaseModel`).
6. **Structured Outputs:** Requires setting `response_format` and explicitly requesting JSON format in the system prompt.
7. **Production Prompts:** Define Role, Task, Constraints, Format, Examples, and Fallback.
8. **ReAct Loop:** Thought $\rightarrow$ Action $\rightarrow$ Observation cycle enabling tool execution.
9. **Prompt Chaining:** Deterministic sequence of prompts, offering easy debugging and cost optimization.
10. **Perceived Latency:** Improved via streaming (`stream=True`), showing users tokens incrementally.
11. **RAG:** Context-driven generation using document retrieval to stop hallucinations.
12. **Vector DB:** Enables semantic search using vector indexes like HNSW, metadata filters, and tenant segregation.